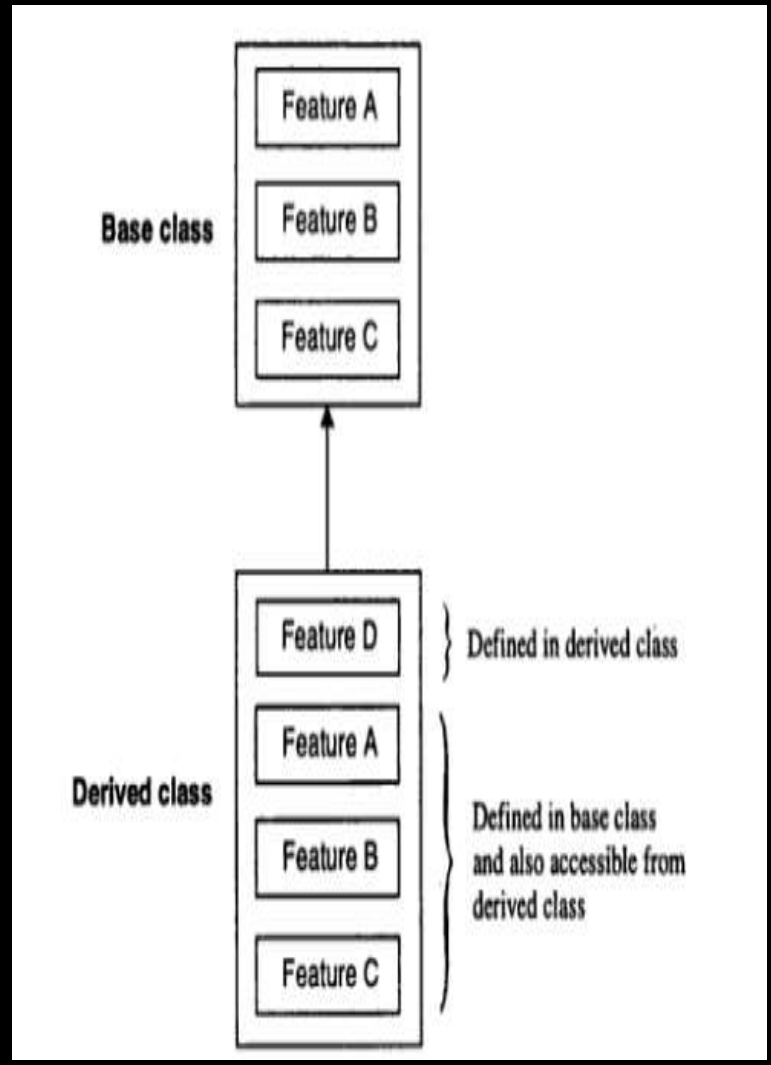
UNIT III
INHERITANCE AND VIRTUAL FUNCTIONS

TOPICS

- Declaration of derived class, forms of inheritance, constructors and destructors in derived class, types of inheritance, abstract class, Virtual functions: Need of virtual functions, Pointers to derived class objects, pure virtual functions, Virtual destructors, Rules of writing virtual function.

Inheritance

- **Inheritance** in C++ is an object-oriented programming (OOP) concept that allows one class (called the **derived class**) to inherit the properties and behaviors (data members and member functions) from another class (called the **base class**).
- Inheritance helps to promote **code reusability**, allowing the derived class to use or modify the features of the base class without having to rewrite the code.



Inheritance

- A base class is often called the ancestor, parent, or superclass, and a derived class is called the descendent, child, or subclass.
- A derived class may itself be a base class from which additional classes are derived. There is no specific limit on the number of classes that may be derived from one another, which forms a class hierarchy.
- C++ provides three access specifiers private, protected, public. They control the visibility and accessibility of class members.
- Access specifier control who can access the data members and member functions of a class.

Inheritance

Advantages of Inheritance are:

- Reusing existing code
- Faster development time
- Easy to maintain
- Easy to extend
- Memory Utilization

Derived Class Declaration

- A derived class is a new class created from an existing base class.
- It inherits accessible properties and functions of the base class.
- It can add new features of its own.

Syntax:

```
class DerivedClass : [VisibilityMode] BaseClass
{
    // members of derived class
};
```

- The derived class declaration specifies:
  - class - Keyword
  - DerivedClass - Name of the derived class
  - : -Shows the derivation from the base class.
  - VisibilityMode - Specifies the type of derivation
  - BaseClass-Name of the base class.

Visibility mode

- **Public:** When a base class is inherited using public, its public members remain public, and its protected members remain protected in the derived class

Syntax:

```
class Derived : public Base
{
};
```

- **Private:** When a base class is inherited using private, its public and protected members become private members of the derived class.

Visibility mode

Syntax:

```
class Derived : private Base
{
};
```

Visibility of Base Class Members in Different Types of Inheritance

Base class member (Visibility mode)	Derived class member visibility		
	Public derivation (public inheritance)	**Private derivation** (private inheritance)	**Protected derivation** (protected inheritance)
public	public	private	protected
private	Not inherited	Not inherited	Not inherited
protected	protected	private	protected

Note: Private members of the base class are never inherited in any type of inheritance.

- **Protected:** When a base class is inherited using protected, its public and protected members become protected members of the derived class.

Syntax:

```
class Derived : protected Base
{
};
```

Public Inheritance

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void eat()
    {
        cout << "This animal eats food." << endl;
    }

protected:
    void sleep()
    {
        cout << "This animal sleeps." << endl;
    }
private:
    void breathe()
    {
        cout << "This animal breathes." << endl;
    }
};

class Dog : public Animal
{
public:
    void display()
    {
        eat();      // ? Public member
        sleep();    // ? Protected member
        // breathe(); // ? Error: private member
    }
};
```

Public Inheritance

```
int main()
{
    Dog myDog;

    myDog.display();

    myDog.eat();     // ? Accessible: eat() is public in Dog
    // myDog.sleep();  // ? Error: sleep() is protected

    return 0;
}
```

Protected Inheritance

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void eat()
    {
        cout << "This animal eats food." << endl;
    }

protected:
    void sleep()
    {
        cout << "This animal sleeps." << endl;
    }

private:
    void breathe()
    {
        cout << "This animal breathes." << endl;
    }
};

class Dog : protected Animal
{
public:
    void display()
    {
        eat();      // ? Accessible inside Dog
        sleep();    // ? Accessible inside Dog
        // breathe(); // ? Error: private member
    }
};
```

Protected Inheritance

```
int main()
{
    Dog myDog;

    myDog.display();

    // myDog.eat();    // ? Error: eat() is protected
    // myDog.sleep();  // ? Error: sleep() is protected

    return 0;
}
```

Private Inheritance

```
#include <iostream>
using namespace std;

class Animal
{
public:
    void eat()
    {
        cout << "This animal eats food." << endl;
    }

protected:
    void sleep()
    {
        cout << "This animal sleeps." << endl;
    }
private:
    void breathe()
    {
        cout << "This animal breathes." << endl;
    }
};

class Dog : private Animal
{
public:
    void display()
    {
        eat();      // ? Accessible inside Dog
        sleep();    // ? Accessible inside Dog
        // breathe(); // ? Error: private member
    }
};
```

Private Inheritance

```
int main()

{
    Dog myDog;

    myDog.display();

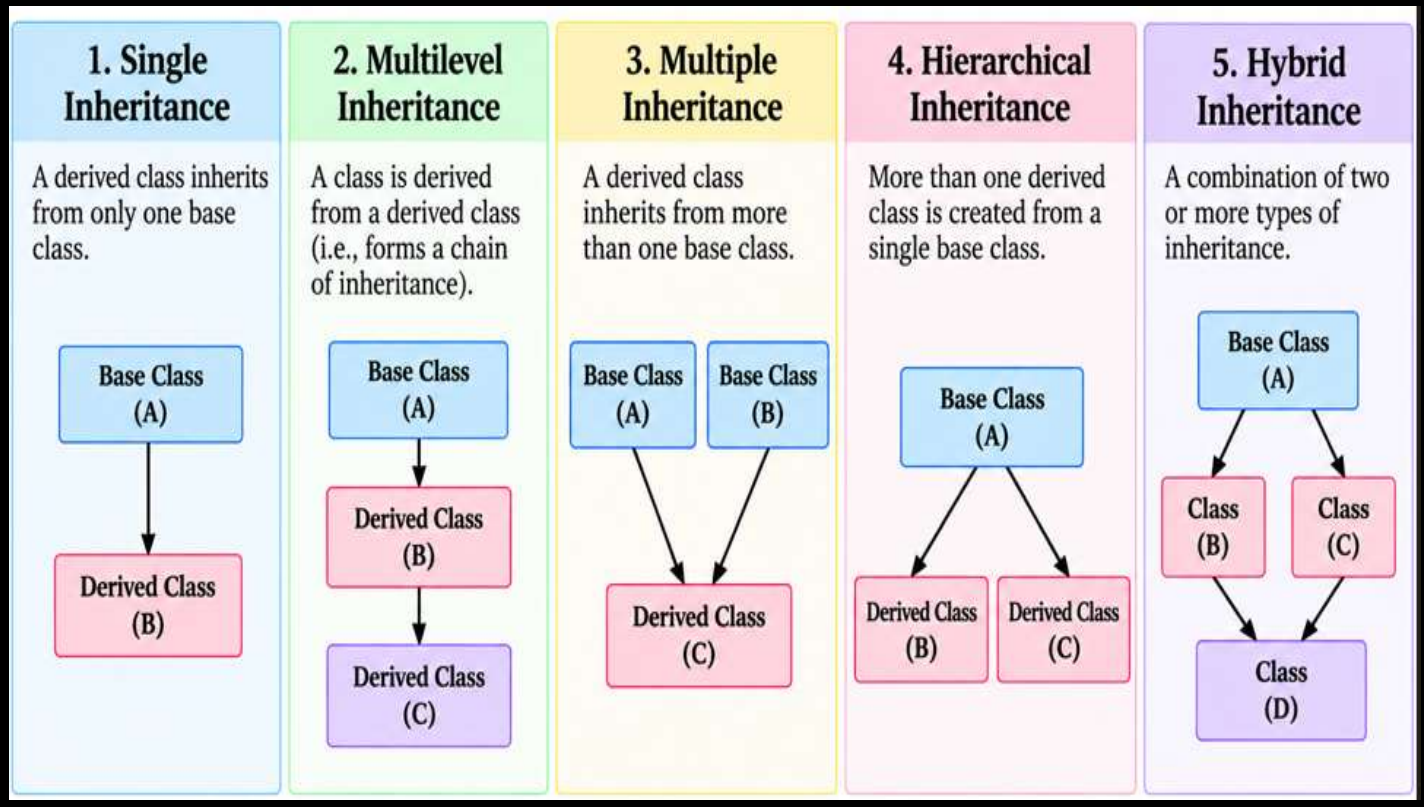
    // myDog.eat();    // ? Error: eat() is private in Dog
    // myDog.sleep();  // ? Error: sleep() is private in
Dog

    return 0;
}
```

Types of Inheritance

Based on the relationship, inheritance can be classified into five forms:

- Single Inheritance
- Multilevel Inheritance
- Multiple Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance



Single Inheritance

- Single Inheritance occurs when one derived class inherits from only one base class.

Syntax:

```
class Base_Class
{
 ……….
 };
class Derived_class : public Base_calss
{
……….
};
```

Multilevel Inheritance

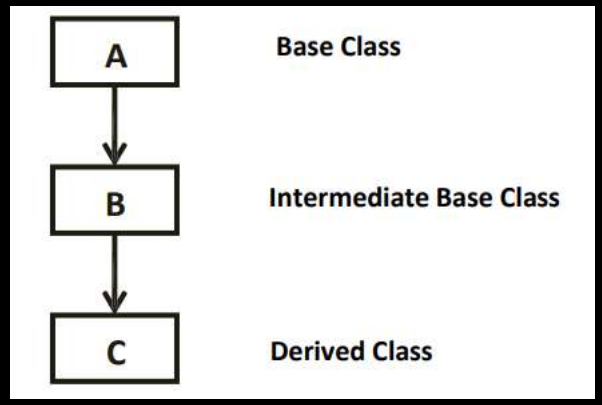
- The mechanism of deriving a derived class from another derived class is known as Multilevel Inheritance.

Syntax:

```
class A
{
   .....
};

class AB : public A
{
   .....
};

class ABC : public AB
{
   .....
};
```



Multilevel Inheritance

```
#include <iostream>
using namespace std;

class A
{
public:
    int a;

    void geta()
    {
        cout << "\nEnter the value of A: ";
        cin >> a;
    }
};
```

```
class B : public A
{
public:
    int b;


    void getb()
    {
        cout << "\nEnter the value of B:
";
        cin >> b;
    }
};
```

```
class C : public B

{
public:
   int c;

   void getc()
   {
      cout << "\nEnter the value of C: ";
      cin >> c;
   }

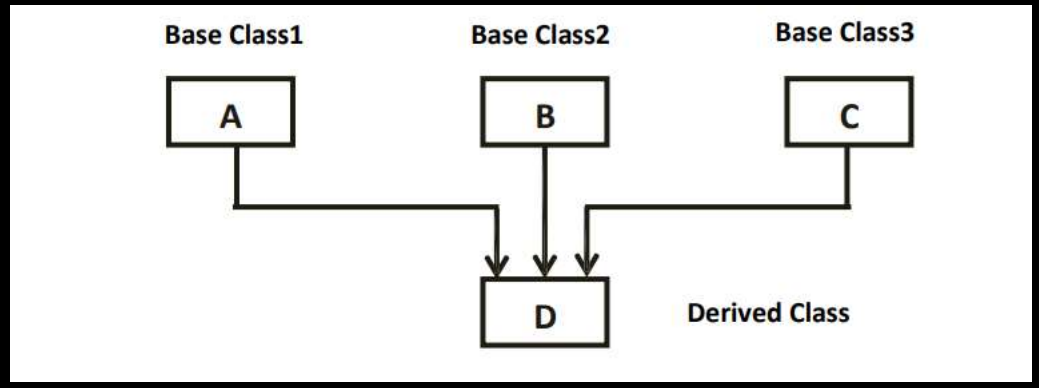
   void display()
   {
      cout << "\nA : " << a;
      cout << "\nB : " << b;
      cout << "\nC : " << c;
   }
};
int main()
{
   C obj;

   obj.geta();
   obj.getb();
   obj.getc();
   obj.display();

   return 0;
}
```

Multiple Inheritance

- Multiple inheritance occurs when one derived class inherits from two or more base classes.



Syntax:

```
class A              // Base Class A
{
   .....
};

class B              // Base Class B
{
   .....
};

class C              // Base Class C
{
   .....
};

class Derived_Class : public A, private B, protected C
{
   .....         // Members of derived class
};
```

Multiple Inheritance

```
#include <iostream>
using namespace std;

class A
{
public:
   int a;

   void geta()
   {
      cout << "\nEnter the value of A : ";
      cin >> a;
   }
};
```

```
class B
{
public:
   int b;

   void getb()
   {
      cout << "\nEnter the value of B : ";
      cin >> b;
   }
};
```

```
class C : public A, public B
{
public:
    int c;

    void getc()
    {
        geta();
        getb();

        cout << "\nEnter the value of C : ";
        cin >> c;
    }

    void display()
    {
        cout << "\n\nA : " << a;
        cout << "\nB : " << b;
        cout << "\nC : " << c;
    }
};

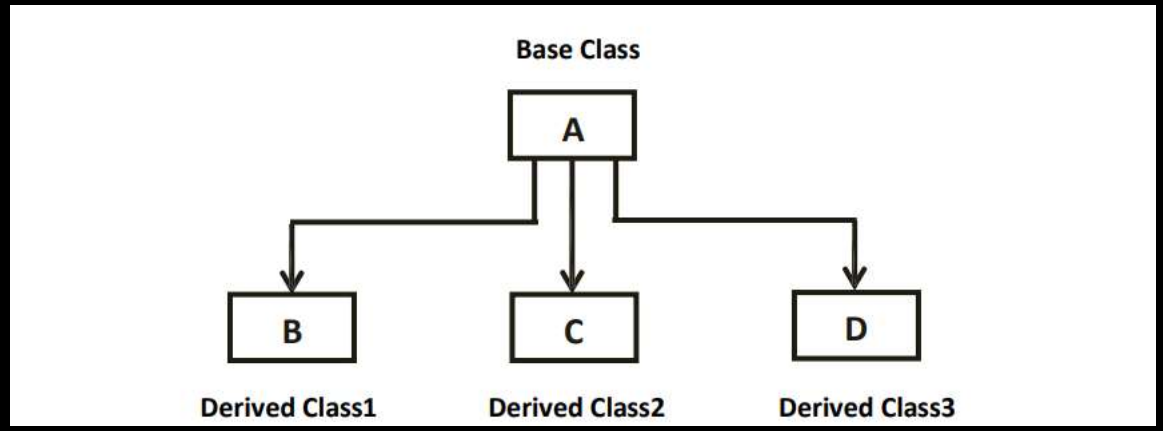
int main()
{
    C obj;

    obj.getc();
    obj.display();

    return 0;
}
```

Hierarchical Inheritance

- The mechanism of deriving multiple derived classes from only base class is known as Hierarchical inheritance.



- Syntax:

```
class Base
{
   // members
};

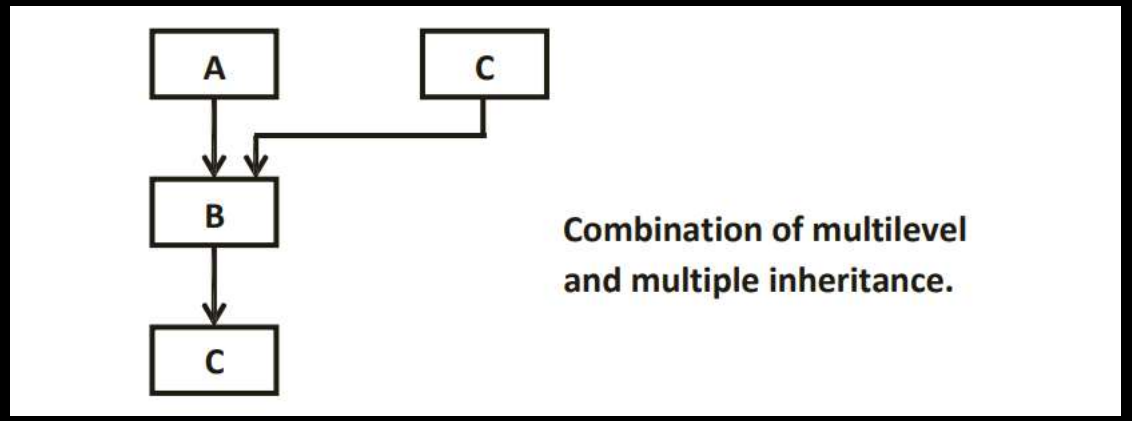
class Derived1 : public Base
{
   // members
};

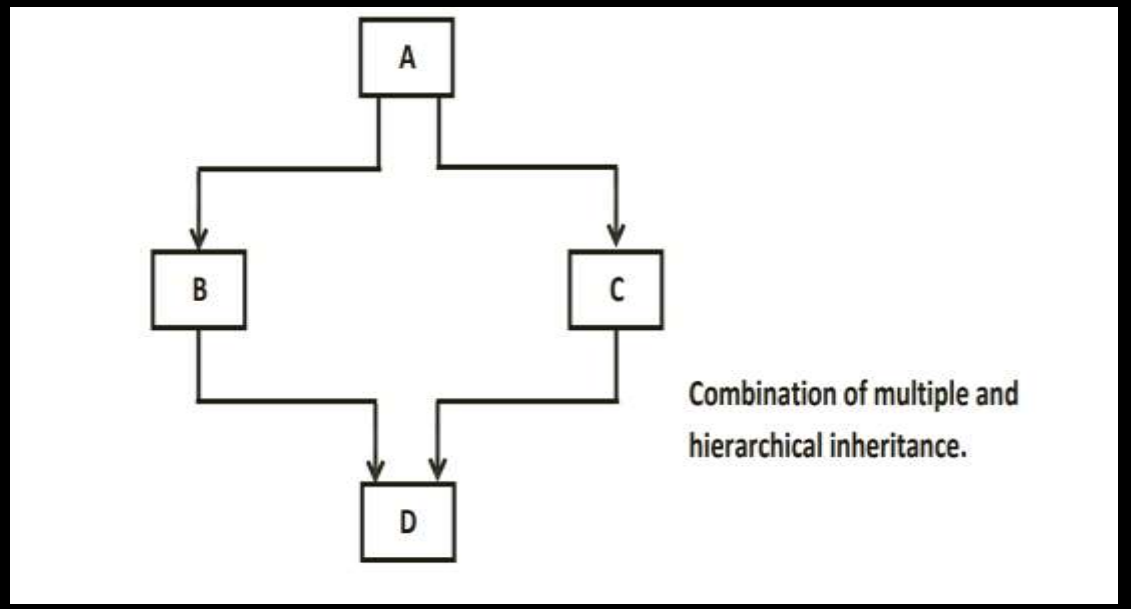
class Derived2 : public Base
{
   // members
};

class Derived3 : public Base
{
   // members
};
```

Hybrid Inheritance

- Combination of two or more types of inheritances is known as hybrid inheritance.





Hybrid Inheritance

```
#include <iostream>
using namespace std;

class Student
{
public:
   int roll;
   char name[20];

   void getdata()
   {
      cout << "\nEnter the roll number of student : ";
      cin >> roll;
      cout << "\nEnter the name of student : ";
      cin >> name;
   }
};

class Test : public Student
{
public:
   int m1, m2;

   void accept()
   {
      cout << "\nEnter the mark of first subject : ";
      cin >> m1;

      cout << "\nEnter the mark of second subject : ";
      cin >> m2;
   }
};
```

```
class Sport
{
public:
   float score;

   void getinfo()
   {
      cout << "\nEnter the score achieved in sport : ";
      cin >> score;
   }
};
class Result : public Test, public Sport
{
public:
   float total;

   void calculate()
   {
      total = m1 + m2 + score;
   }

   void display()
   {
      cout << "\nRoll Number = " << roll;
      cout << "\nName = " << name;
      cout << "\nMarks of Subject 1 = " << m1;
      cout << "\nMarks of Subject 2 = " << m2;
      cout << "\nSports Score = " << score;
      cout << "\nTotal = " << total;
   }
};
```

```
int main()
{
    Result obj;

    obj.getdata();
    obj.accept();
    obj.getinfo();
    obj.calculate();
    obj.display();

    return 0;
}
```

Constructors And Destructors In Derived Class

- The derived class does not need to have a constructor as long as the base class has a no-argument constructor. However, if the base class has constructors with arguments (one or more), then the derived class must have a constructor and pass the arguments to the base class constructor.

- Invocation of constructors and destructors depends on the type of inheritance being implemented.

- When an object of a derived class is created, the constructor of the base class is executed first, and then the constructor of the derived class is executed.

Constructors And Destructors In Derived Class

- A destructor is special member function that is executed when an object of that class is destroyed.
- Destroying an object means, de-allocating all the resources such as memory that was allocated for the object by the constructor.
- It will have like constructor, the name same as that of the class but preceded by a tilde (~).
- Destructors are called in the reverse sequence of constructor invocation, i.e., the destructor of the derived class is called first, and the destructor of the base class is called next.

Constructors in Derived Class

- **No constructor in the derived and base class**

When there are no constructors either in the base or derived classes, the compiler automatically creates objects of classes without any error when the class is instantiated.

Constructors in Derived Class

- **No constructor in the derived and base class**

```
class B // Base class
{
    // Body of base class, without constructors
};

class D : public B // Publicly derived class
{
    // Body of derived class, without constructors
public:
 void msg()
    {
        cout << "No constructors exist in base and derived class" << endl;
    }
};

int main()
{
    D objd; // Object of derived class
    objd.msg();

    return 0;
}
```

Constructors in Derived Class

- **Constructor only in the base class**

```
#include <iostream>
using namespace std;

class B  // Base class
{
public:
    B()
    {
        cout << "No-argument constructor of the base class B is executed";
    }
};
 class D : public B  // Publicly derived class
{
public:
};

int main()
{
    D obj1;  // Calls the base class constructor automatically
    return 0;
}
```

Constructors in Derived Class

- **Constructor only in the derived class**

```
class B  // Base class
{
//body of base class, without constructor
};
 class D : public B  // Publicly derived class
{
public:
D()
{
    cout<< “constructor only exist in derived class”;
}
};

int main()
{
   D obj1; // accesses derived constructor
return 0;
}
```

Constructors in Derived Class

- **Constructor in both base and derived class**

```
class B  // Base class
{
public:
    B()
    {
        cout << "No-argument constructor of the base class B executed first\n";
    }
};

class D : public B  // Publicly derived class
{
public:
    D()
    {
        cout << "No-argument constructor of the derived class D executed next\n";
    }
};

int main()
{
    D objd;  // Calls both base and derived class constructors
    return 0;
}
```

Constructors in Derived Class

- **Multiple Constructors in base class and single constructor in derived class**

```
class B  // Base class
{
public:
    B()
    {
        cout << "No-argument constructor of the base class B";
    }

    B(int a)
    {
        cout << "One-argument constructor of the base class B";
    }
};
```

```
class D : public B  // Publicly derived class
{
public:
    D(int a)
    {
        cout << "\nOne-argument constructor of the derived class D";
    }
};

int main()
{
    D objd(3);
    return 0;
}
```

Constructors in Derived Class

- **Constructor in Base and Derived Classes Without Default Constructor:**

If the base class does not have a default constructor and has only a parameterized constructor, it must be explicitly invoked by the derived class. Otherwise, the compiler generates an error.

Constructors in Derived Class

```
class B  // Base class
{
public:
    B(int a)
    {
        cout << "One-argument constructor of
the base class B";
    }
};

class D : public B  // Publicly derived class
{
 public:
    D(int a)
    {
        cout << "\nOne-argument
constructor of the derived class D";
    }
};

int main()
{
    D objd(3);
    return 0;
}
// compilation error: Cannot find
default constructor to initialize base
class B
```

Constructors in Derived Class

- **Explicit Invocation in the Absence of a Default Constructor:**

If the base class has a parameterized constructor but no default constructor, the derived class must call the base class constructor explicitly. Otherwise, the compiler generates an error because it cannot find a default constructor. This can be done using a constructor initializer list.

Example: `D(int a) : B(a)`

Constructors in Derived Class

```
class B  // Base class
{
public:
    B(int a)
    {
        cout << "One-argument constructor of the
base class B";
    }
};

class D : public B  // Publicly derived class
{
public:
  D(int a) : B(a)
    {
        cout << "\nOne-argument
constructor of the derived class D";
    }
};

int main()
{
    D objd(3);
    return 0;
}
```

Constructors in Derived Class

- **Constructor in a Multiple Inherited Class with Default Invocation**

```
class B1  // Base class
{
public:
   B1()
   {
      cout << "\nNo-argument constructor of the base class B1";
   }
};

class B2  // Base class
{
public:
   B2()
   {
      cout << "\nNo-argument constructor of the base class B2";
   }
};

class D : public B2, public B1  // Multiple inheritance
{
public:
   D()
   {
      cout << "\nNo-argument constructor of the derived class D";
   }
};

int main()
{
   D objd;
   return 0;
}
```

Constructors in Derived Class

- **Constructor in a Multiple Inherited Class with Explicit Invocation**

```
class B1  // Base class
{
public:
   B1()
   {
      cout << "\nNo-argument constructor of the base class B1";
   }
};

class B2  // Base class
{
public:
   B2()
   {
     cout << "\nNo-argument constructor of the base class B2";
   }
};

class D : public B2, public B1  // Multiple inheritance
{
public:
   D() : B2(), B1()  // Explicit invocation of constructors
   {
      cout << "\nNo-argument constructor of the derived class D";
   }
};

int main()
{
   D objd;
   return 0;
}
```

Constructors in Derived Class

- **Constructor in Base and Derived Classes in Multiple Inheritance**

```
class B1  // Base class
{
public:
   B1()
   {
      cout << "\nNo-argument constructor of the base class B1";
   }
};

class B2  // Base class
{
public:
   B2()
   {
 cout << "\nNo-argument constructor of the base class B2";
   }
};

class D : public B1, virtual public B2
{
public:
   D() : B1(), B2()
   {
      cout << "\nNo-argument constructor of the derived class D";
   }
};

int main()
{
   D objd;
   return 0;
}
```

Constructors in Derived Class

- **Constructor in Multilevel Inheritance**

```
class B  // Base class
{
public:
   B()
   {
      cout << "\nNo-argument constructor of the base class B";
   }
};

class D1 : public B  // Derived class
{
public:
   D1()
   {
      cout << "\nNo-argument constructor of the class D1";
   }
};
```

```
class D2 : public D1  // Further derived class
{
public:
   D2()
   {
      cout << "\nNo-argument constructor of the derived class D2";
   }
};

int main()
{
   D2 objd;
   return 0;
}
```

Constructors in Derived Class

- When both base and derived classes have constructors, the base class constructor executes first, followed by the derived class constructor.
- **Multiple inheritance**: Base class constructors execute in the order in which the base classes are declared in the inheritance list.
- **Multilevel inheritance:** Constructors execute from the topmost base class to the last derived class.
- The derived class receives the required arguments and passes the appropriate values to the base class constructors using the constructor initializer list.

Constructors in Derived Class

- Order of Invocation of Constructors

Method of Inheritance	Order of Execution

`class D: public B`
`{`
`...`
`};`
	B( ): base constructor
	D( ): derived constructor

`class D: public B1, public B2`
`{`
`...`
`};`
	B1( ): base constructor
	B2( ): base constructor
	D( ): derived constructor

`class D: public B1, virtual B2`
`{`
`..`
`};`
	B2( ): virtual base constructor
	B1( ): base constructor
	D( ): derived constructor

`class D1: public B`
`{`
`...`
`};`
`class D2: public D1`
`{`
`..`
`};`
	B( ): super base constructor
	D1( ): base constructor
	D2( ): derived constructor

Destructors in Derived Classes

- If a destructor is not explicitly defined in a class, the compiler provides an implicit destructor, which is invoked automatically when the object is destroyed.

```
class B1  // Base class
{
public:
   B1()
   {
      cout << "\nNo-argument constructor of the base class B1";
   }

   ~B1()
   {
      cout << "\nDestructor in the base class B1";
   }
};

class B2  // Base class
{
public:
   B2()
 {
      cout << "\nNo-argument constructor of the base class B2";
   }
~B2()
   {
cout << "\nDestructor in the base class B2";
   }
};

class D : public B1, public B2  // Derived class
{
public:
   D()
   {
      cout << "\nNo-argument constructor of the derived class D";
   }

   ~D()
   {
      cout << "\nDestructor in the derived class D";
   }
};

int main()
{
   D objd;
   return 0;
}
```

Virtual Base Class

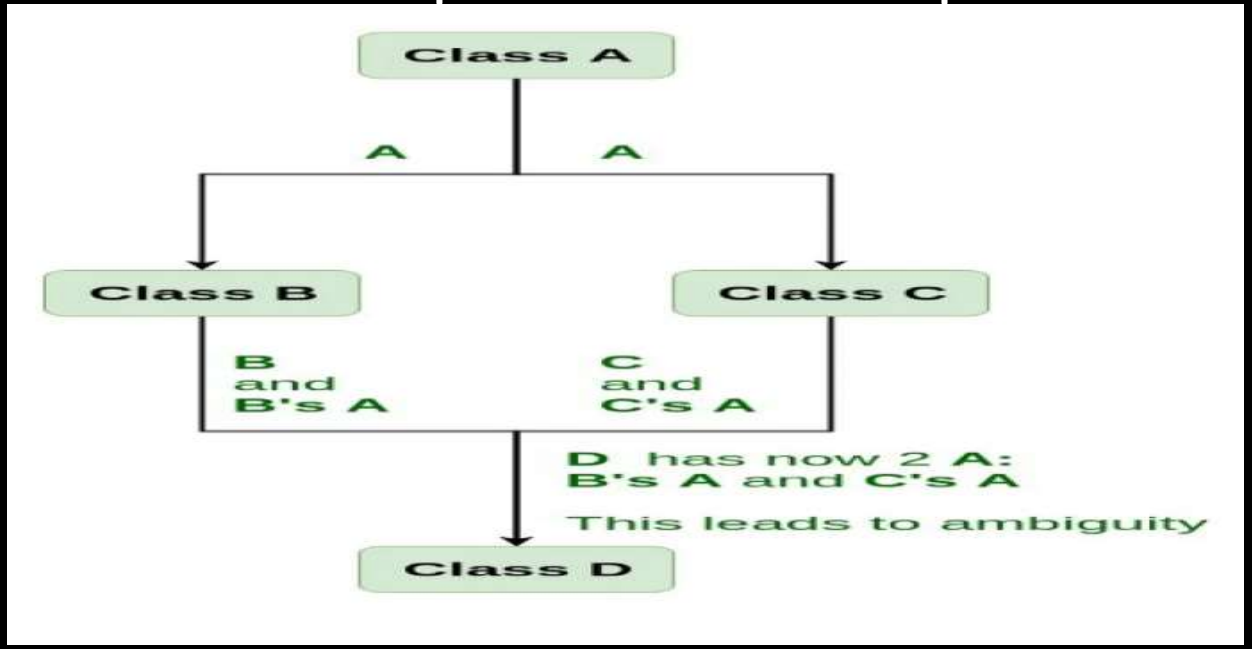
- Virtual base classes are used in multiple inheritance to prevent multiple copies of the same base class from appearing in the inheritance hierarchy.

- They are declared using the keyword **virtual** before or after the access specifier.

- Virtual base class avoids ambiguity and ensures that only one shared copy of the base class is inherited through multiple inheritance paths.

Need for Virtual Base Class

- Consider a situation where we have a class A, which is inherited by two classes, B and C. Both classes B and C are then inherited by another class, D, forming a diamond-shaped inheritance hierarchy. This is known as the diamond problem in multiple inheritance.



Example of Diamond problem

```
class A
{
public:
   int x;

   A()
   {
 x = 10;
   }
};
class B : public A
{
};
```

```
class C : public A
{
};

class D : public B, public C
{
};

int main()
{
   D obj;

   cout << obj.x;  // Compilation error: ambiguous member

   return 0;
}
```

How to resolve ambiguity

- To resolve this ambiguity when class **A** is inherited in both class **B** and class **C**, it is declared as **virtual base class** by placing a keyword **virtual** as :

Syntax 1:

```
class B : virtual public A
{
};
```

Syntax 2:

```
class C : public virtual A
{
};
```

Virtual Base Class(Example)

```
class Vehicle
{
public:
    int speed;

    Vehicle()
    {
        speed = 60;
    }
};

class Car : virtual public Vehicle
{
};

class Bike : virtual public Vehicle
{
};

class HybridVehicle : public Car, public Bike
{
};

int main()
{
    HybridVehicle h;

    cout << "Vehicle speed: " << h.speed << endl;

    return 0;
}
```

Pointers to derived class objects

- In C++, a pointer to a base class can point to an object of the base class as well as an object of its derived class.
- It can directly access members available through the base-class interface.

•It cannot directly access members declared only in the derived class.

•A non-virtual function call through the base pointer uses the base-class version when the function is hidden or redefined in the derived class.

•To access members specific to the derived class, we can use a pointer to the derived class.

Pointers to derived class objects

```
class Base
{
public:
    int x = 10;

    void show()
    {
        cout << "Base class x = " << x << endl;
    }
};

class Derived : public Base
{
public:
    int y = 20;
};
```

```
int main()
{
    Derived obj;
    Base *ptr = &obj;

    cout << "x = " << ptr->x << endl;

    // Access derived class member using derived pointer
    Derived *dptr = &obj;
    cout << "y = " << dptr->y << endl;

    return 0;
}
```

Virtual function

- Virtual functions support runtime polymorphism.
- They enable dynamic binding or late binding, in which the function call is resolved at runtime.
- A virtual function is a member function declared in the base class using the **virtual** keyword.
- It can be overridden in a derived class to provide a different implementation.

Virtual function

- When a function is declared virtual, C++ selects the appropriate overridden function at runtime based on the actual type of the object pointed to by the base class pointer.

- Without a virtual function, a function call through a base class pointer normally invokes the base class version.

Virtual function

```
class Media
{
protected:
    string title, publication;

public:
    virtual void read()
    {
        cout << "Enter title: ";
        getline(cin, title);
        cout << "Enter publication: ";
        getline(cin, publication);
    }

    virtual void show()
    {
        cout << "Title: " << title << endl;
        cout << "Publication: " << publication << endl;
    }

    virtual ~Media() {}
};
```

```
class Book : public Media
{
private:
    int pages;

public:
    void read() override
    {
        Media::read();

        cout << "Enter number of pages: ";
        cin >> pages;
        cin.ignore();
    }
    void show() override
    {
        Media::show();
        cout << "Number of pages: " << pages << endl;
    }
};

class Tape : public Media
{
private:
    float playingTime;

public:
    void read() override
    {
        Media::read();
```

```
 cout << "Enter playing time in minutes: ";
        cin >> playingTime;
        cin.ignore();
    }

    void show() override
    {
        Media::show();
        cout << "Playing time: " << playingTime
             << " minutes" << endl;
    }
};

int main()
{
    Media *ptr;
    Book b;
    Tape t;
    cout << "Enter book details:\n";
    ptr = &b;
    ptr->read();

    cout << "\nEnter tape details:\n";
    ptr = &t;
    ptr->read();

    cout << "\nBook Details:\n";
    ptr = &b;
    ptr->show();

    cout << "\nTape Details:\n";
    ptr = &t;
    ptr->show();

    return 0;
}
```

Rules of writing virtual function

- Virtual functions must be member functions of a class.
- They cannot be static member functions.
- They are commonly accessed through base class pointers or references.
- A virtual function can be a friend of another class, but a friend function itself cannot be virtual.
- A virtual function declared in a base class generally needs a definition if it is called or otherwise requires one.

Rules of writing virtual function

- The base and derived class versions must have compatible function signatures to override correctly.

- Constructors cannot be virtual, but destructors can be virtual.

- A base class pointer can point to a derived class object, but a derived class pointer cannot directly point to a base class object without an explicit cast.

- A virtual function does not need to be overridden in every derived class. If it is not overridden, the inherited base class implementation is used.

Virtual Destructor

- A virtual destructor is declared using the ***virtual*** keyword in the base class. It ensures that when a derived class object is deleted through a base class pointer, the derived class destructor executes first, followed by the base class destructor.

Why We Use Virtual Destructor

- They ensure the proper destruction of derived class objects through base class pointers.

Virtual Destructor

- They help release resources allocated by both base and derived classes.
- They prevent resource leaks and undefined behavior when deleting derived objects through base class pointers.
- They support runtime binding during object destruction.
- Constructors cannot be virtual, but destructors can be virtual.
- If the base class destructor is virtual, the derived and base class destructors execute in the correct order

Virtual Destructor

```
#include <iostream>
using namespace std;

class Father
{
public:
    virtual ~Father()
    {
        cout << "Father destructor" << endl;
    }
};
class Son : public Father
{
public:
    ~Son()
    {
        cout << "Son destructor" << endl;
    }
};

int main()
{
    Father* ptr = new Son();

    delete ptr;

    return 0;
}
```

Abstract Class and Pure Virtual Function

- **abstract class :**
- An **abstract class** is one that is not used to create objects.

- It is designed to be specifically used as a base class.

- It contains at least one *pure virtual function*.

- It is used when a class has only a partial implementation of its methods.

Pure Virtual Function and Abstract Class

- It promote code reusability and reduce duplication.
- It defines common features and allows derived classes to extend or modify functionality according to their requirements.

Pure Virtual Function and Abstract Class

pure virtual function:

- A **pure virtual function** is also known as an abstract function.
- A pure virtual function generally has no implementation in its declaration.
- The derived class must implement all inherited pure virtual functions to become a concrete class; otherwise, it also remains an abstract class.
- Pure virtual functions support runtime polymorphism when overridden in derived classes

Pure Virtual Function and Abstract Class

- A pure virtual function is declare using a pure specifier = 0 in the declaration of a virtual member function within a class.

Example:

```
class AB
 {
public:
        virtual void f() = 0;
};
```

```
// Abstract base class
class Shape
{
public:
    virtual double area() = 0;
};

// Derived class: Circle
class Circle : public Shape
{
    double radius;

public:
    Circle(double r)
    {
        radius = r;
    }
    double area()
    {
        return 3.14159 * radius * radius;
    }
};

// Derived class: Rectangle
class Rectangle : public Shape
{
    double length, width;

public:
    Rectangle(double l, double w)
    {
        length = l;
        width = w;
    }
```

```
    double area()
    {
        return length * width;
    }
};

// Derived class: Triangle
class Triangle : public Shape
{
    double base, height;

public:
    Triangle(double b, double h)
    {
        base = b;
        height = h;
    }
    double area()
    {
        return 0.5 * base * height;
    }
};

int main()
{
    Circle c(5);
    Rectangle r(4, 6);
    Triangle t(4, 3);

    cout << "Area of Circle: " << c.area() << endl;
    cout << "Area of Rectangle: " << r.area() << endl;
    cout << "Area of Triangle: " << t.area() << endl;

    return 0;
}
```

Pure Virtual Function with Parameters

```
// Abstract base class
class Shape
{
public:
    virtual void area(double a, double b) = 0;
};

// Derived class: Circle
class Circle : public Shape
{
public:
    void area(double a, double b) override
    {
        cout << "Area of Circle: " << 3.14159 * a * a <<
endl;
    }
};
```

```
// Derived class: Rectangle
class Rectangle : public Shape
{
public:
    void area(double a, double b) override
    {
        cout << "Area of Rectangle: " << a * b << endl;
    }
};

// Derived class: Triangle
class Triangle : public Shape
{
public:
```

```
 void area(double a, double b) override
    {
        cout << "Area of Triangle: " << 0.5 * a * b << endl;
    }
};

int main()
{
    Circle c;
    Rectangle r;
    Triangle t;

    c.area(6, 0);  // Radius = 6
    r.area(7, 4);  // Length = 7, Width = 4
    t.area(6, 3);  // Base = 6, Height = 3

    return 0;
}
```

Object composition

- Object composition is an object-oriented programming technique in which an object contains one or more objects of other classes as its data members.

- Composition represents a has-a relationship between objects.

Object composition

Why is composition needed?

- Suppose we want to create a Car class. A car needs an engine to start, but the engine has its own properties and operations.

- We could write all engine-related code inside the Car class. However, this would combine two different responsibilities into one class.

- Instead, we can create:
  - An Engine class to manage engine operations.
  - A Car class that contains an Engine object.

Object composition

Syntax:

```
class Component {
public:
    void display() {
        // Component operation
    }
};

class Whole {
private:
    Component obj;  // Composition

public:
    void show() {
        obj.display();
    }
};
```

Object composition

```
#include <iostream>
using namespace std;

class Engine {
public:
    void start() {
        cout << "Engine started." << endl;
    }

    void stop() {
        cout << "Engine stopped." << endl;
    }
};
class Car {
private:
    Engine engine;  // Composition: Car has an Engine
public:
    void startCar() {
        cout << "Starting the car..." << endl;
        engine.start();
    }

    void stopCar() {
        cout << "Stopping the car..." << endl;
        engine.stop();
    }
};

int main() {
    Car c1;

    c1.startCar();
    c1.stopCar();

    return 0;
}
```

Object delegation

- Object delegation is an object-oriented programming technique in which one object assigns a task or responsibility to another object by calling its member functions.

- Instead of doing all the work by itself, an object takes help from another object to complete a task.

Object delegation

- **Syntax:**

```
class Helper {
public:
    void performTask() {
        // Task implementation
    }
};

class Manager {
private:
    Helper helper;

public:
    void requestTask() {
        helper.performTask();
    }
};
```

Object delegation

```
#include <iostream>
using namespace std;

class Result {
private:
    int marks;

public:
    Result(int m) {
        marks = m;
    }

    void displayMarks() {
        cout << "Marks: " << marks << endl;
    }
};
class Student {
private:
    int rollNo;
    Result result;
public:
    Student(int r, int m) : rollNo(r), result(m) {}

    void displayDetails() {
        cout << "Roll Number: " << rollNo << endl;

        // Delegation
        result.displayMarks();
    }
};

int main() {
    Student s1(101, 85);
    s1.displayDetails();

    return 0;
}
```